
CS1004 - Object Oriented Programming

Semester Project : Deadline : 03rd-May-2026 5:00 PM

IMAGE FILTER STUDIO

Instructions:

- Submit a .zip containing files only with you name (**i21-XXXX_Name_Section.zip**).
- Strict plagiarism policy applies to the code submitted.
- Viva will be taken for all submissions.
- Non-running code will be awarded zero marks.
- This is an individual assignment.
- There are 2 phases for this project. Phase 1 is a class diagram.

1. Project Overview

You are building a console-based C++ Image Filter Studio. The program works with real image files. A user loads a real image (JPG or PNG format), the program reads and decodes it into a pixel grid, applies a pipeline of image filters, and writes the processed image back to disk : a real image the user can open in any viewer.

The project must demonstrate every OOP concept taught throughout the course : from raw pointers to polymorphism, from encapsulation to virtual functions. Think of it as your OOP portfolio project.

The Four Big Ideas

1. Real Image I/O : Load a JPG/PNG image from disk, decode pixels; write the result back.
2. Pixel Grid : An image is a 2D array of Pixel objects, dynamically allocated on the heap.
3. Filter Pipeline : Filters chain together; each runs on the output of the previous one.
4. OOP Showcase : Every concept from lab week 2 through week 9 must appear somewhere.

1a. User Roles

Role	What They Can Do
Admin	Manages the filter catalog (enable / disable), manages customer accounts, views all sessions

Role	What They Can Do
Customer	Registers, logs in, loads a real JPG/PNG image, selects filters, applies them, saves the result, views history

2. Real Image I/O : JPG / PNG Format

2a. What is a JPG / PNG File?

JPG and PNG are the most widely used real image formats. JPG uses lossy compression for photographs; PNG uses lossless compression and supports transparency. Both store RGB pixel data and can be opened in any image viewer or editor.

There are two variants you should support:

- JPG : lossy compression, best for photographs, small file sizes.
- PNG : lossless compression, best for graphics with sharp edges, supports transparency.

2b. JPG Format

JPG files are read/written using a library such as stb_image / stb_image_write (single-header, public domain):

```
stbi_image_free(pixels);
# optional comment line (skip lines starting with #)
// Load a JPG (or PNG) file:
int width, height, channels;
unsigned char* pixels = stbi_load(path, &width, &height, &channels, 3);
unsigned char* pixels = stbi_load(path, &width, &height, &channels, 3);
// pixels[3*(y*width+x)+0] = R,  [+1] = G,  [+2] = B
```

// Free when done:

```
stbi_image_free(pixels);

// channels will be 3 (RGB). stbi_load returns null on failure.
```

2c. PNG Format

PNG files can be read and written using the same stb_image / stb_image_write library. PNG is lossless and supports an alpha (transparency) channel, though for this project only the RGB channels are required.

```
#define STB_IMAGE_WRITE_IMPLEMENTATION
#include "stb_image_write.h"
// Write PNG:
```

Sample console preview (20 x 10 image):

Image size: 20 x 10 | Total pixels: 200

4. The Filter Catalog : 10 Filters

4a. Overview

Filters are the heart of the project. You must implement at minimum 10 filters spanning three categories. Every filter is a class derived from a common abstract base class Filter. The Admin can enable or disable any filter from the catalog. Customers build a pipeline from the currently-enabled filters.

4b. Filter Table

ID	Filter Name	Category	What It Does to Each Pixel
01	Grayscale	Pixel Transform	Replace R,G,B with (R+G+B)/3
02	Invert / Negative	Pixel Transform	newR=255-R, newG=255-G, newB=255-B
03	Brightness Adjust	Pixel Transform	Add a fixed value to every channel; clamp to [0,255]
04	Contrast Stretch	Pixel Transform	Linearly scale each channel: new = (val - min) / (max - min) * 255
05	Red Channel Only	Pixel Transform	Set G=0, B=0; keep R as-is
06	Green Channel Only	Pixel Transform	Set R=0, B=0; keep G as-is
07	Blue Channel Only	Pixel Transform	Set R=0, G=0; keep B as-is
08	Box Blur (3×3)	Spatial Filter	Each pixel = average of itself and its 8 neighbours; handle edges
09	Flip Horizontal	Geometric	Mirror left-to-right: swap pixel[col] with pixel[width-1-col]
10	Flip Vertical	Geometric	Mirror top-to-bottom: swap row[r] with row[height-1-r]

Note on Spatial Filters (08)

This filter looks at neighbouring pixels, not just the current pixel.

For edge pixels where a neighbour does not exist, use only the available neighbours (skip missing ones).

Always operate on a COPY of the image : do not let a modified pixel influence the next pixel's calculation.

4c. catalog.txt Format

```
FilterID|FilterName|Category|IsEnabled
01|Grayscale|Pixel Transform|1
02|Invert|Pixel Transform|1
...
10|Flip Vertical |Geometric|1
```

5. Class Design Guidance

You are responsible for designing the classes, including their names, member variables, and member functions. The descriptions below only provide a few examples of what some classes might look like and outline their general responsibilities; they are not strict definitions. How you structure, name, and implement your classes is entirely up to you. This is intentional, as making these design decisions is an important part of the learning process. You may also add additional classes if you feel something is missing.

Design Rule

Read the responsibility description for each class.

Choose your classes, private member names, and method names.

Your design will be evaluated on correctness, OOP usage,.

6a. The Pixel Class

- Responsibility: Store one pixel's colour (three integer channels, 0–255 each).
- Must have: a static clamp utility function, operator+ for blending, operator<< for ASCII output.
- Data must be private; access via getters and setters.
- Default constructor (black pixel) and parameterised constructor (R, G, B) are both required.

6b. The Image Class

- Responsibility: Store a 2D grid of Pixel objects as a dynamically-allocated double pointer.
- Must have: constructor that allocates the grid, copy constructor that deep-copies it, destructor that frees it.
- Provide an at(row, col) method and const getters for width/height.
- Implement the Saveable interface (write to JPG/PNG file) and displayASCII().
- FilterSession must be declared a friend so it can access the pixel grid directly.

6c. File Manager Classes

- Responsibility: All disk I/O is centralised : do not scatter file operations everywhere.
- One class handles customers.txt (load, save, search, block, delete).
- One class handles catalog.txt (load, save, toggle).
- One class handles sessions.txt (append, load for a specific CNIC).

- One class handles JPG/PNG image reading and writing.

7. Registration, Login, and Account Management

7a. Text File Format

All user data is stored in plain .txt files, one record per line, fields separated by |.

```
customers.txt format:
CNIC|Password|FullName|Gender|Phone|City|IsBlocked

Example:
3520212345679|Ahmed1234|Ahmed Ali|M|03001234567|Lahore|0
```

7b. Registration Rules

Field	Validation Rules
CNIC	Exactly 13 digits. Must be unique (reject duplicates). Must not appear in blocked_cnics.txt.
Password	Exactly 9 characters, at least 1 uppercase letter, at least 1 digit.
Confirm Password	Must match password exactly.
Full Name	Plain text, no special rules.
Gender	M / F / Other
Phone Number	Any format, stored as string.
City	Plain text, no special rules.
IsBlocked	0 or 1. Set to 1 by Admin to ban the account.

7c. Login

- Admin credentials are hardcoded in code. No registration screen for Admin.
- Customer login: CNIC + password. 3 failed attempts = temporary lockout, return to main menu.
- If IsBlocked == 1, deny login with a clear message.

7d. Admin : Customer Management

- View all customers in a formatted table.
- Search customer by CNIC or name.
- Block a customer: set IsBlocked=1 in customers.txt, add CNIC to blocked_cnics.txt.
- Delete a customer record entirely from customers.txt.

How to Delete a Line from a .txt File

You cannot remove a line in-place.
Read all lines into a vector<string>.
Skip the line you want to delete.
Rewrite the entire file from scratch.

8. The Filter Pipeline : Full Workflow

8a. Step 1 : Load a Real Image

The customer provides a path to a JPG or PNG file on disk. The program uses stb_image to load the file, obtaining width, height, and raw RGB pixel data, then fills the Image grid.

```
=== Load Image ===
1. Load from JPG/PNG file
2. Generate test pattern (for testing without a real image)
Choice: 1
Enter path to image file: C:\Users\Ahmed\Desktop\photo.jpg
Loading image file...
Image loaded: 640 x 480 pixels (307,200 pixels total)

=== ASCII Preview (scaled down) ===
:::::::::::::::::=====+++++++
:::::::::::::::::=====+++++++
.....-=====+++++*****#
(truncated for display)

Image ready. Now build your filter pipeline.
```

8b. Step 2 : Build the Pipeline

```
=== Build Filter Pipeline ===
Current pipeline: (empty)

Available Filters:
01  Grayscale           [Pixel Transform]  Enabled
02  Invert              [Pixel Transform]  Enabled
03  Brightness Adjust   [Pixel Transform]  Enabled
04  Contrast Stretch    [Pixel Transform]  Enabled
05  Red Channel Only    [Pixel Transform]  Enabled
06  Green Channel Only  [Pixel Transform]  Enabled
07  Blue Channel Only   [Pixel Transform]  Enabled
08  Box Blur (3x3)      [Spatial Filter]   Enabled
09  Flip Horizontal     [Geometric]         Enabled
10  Flip Vertical       [Geometric]         Enabled

Enter filter ID to add (0 to finish): 1
Added: Grayscale
Pipeline: [ Grayscale ]

Enter filter ID to add (0 to finish): 4
```

```

Enter brightness amount (-100 to +100): 30
Added: Brightness Adjust (+30)
Pipeline: [ Grayscale -> Brightness Adjust(+30) ]

Enter filter ID to add (0 to finish): 0
Pipeline finalised. 2 filters ready to apply.

```

8c. Step 3 : Apply & Preview After Each Filter

```

=== Applying Pipeline ===

Applying filter 1/2: Grayscale ...
::::::::::::::::::::::::::::::::::::
::::::::::::::::::::::::::::::::::::
(colour removed : all channels equal)

Applying filter 2/2: Brightness Adjust (+30) ...
=====
=====
(image visibly brighter)

All 2 filters applied.

```

8d. Step 4 : Save the Result as a Real JPG / PNG

The final processed Image is written back to disk as a JPG or PNG file using `stb_image_write`. The user can open this file in any image viewer to see the result.

```

Pipeline complete.
Save result? (y/n): y
Saving as PNG to: 3520212345679_20260407_143022.png
Image saved successfully.
Session recorded in sessions.txt.

Open the file in any image viewer (Windows Photos, Preview, GIMP) to see your
result.

```

9. All Files You Must Use

File	What It Stores
customers.txt	CNIC Password FullName Gender Phone City IsBlocked
catalog.txt	FilterID FilterName Category IsEnabled
sessions.txt	CNIC Timestamp FiltersApplied OutputFile
blocked_cnics.txt	One banned CNIC per line

File	What It Stores
<CNIC>_<timestamp>.png	The output PNG image from a completed session

sessions.txt example line:

```
3520212345679|20260407_143022|Grayscale>BrightnessAdjust+30|3520212345679_20260407_143022.png
```

10. Menus and Home Screens

10a. Main Startup Screen

```

IMAGE FILTER STUDIO

1. Admin Login
2. Customer Login
3. New Customer? Register here
4. Exit

Your choice: _

```

10b. Admin Panel

```

ADMIN PANEL : Image Filter Studio

1. Manage Filter Catalog
2. Manage Customers
3. View All Sessions
4. Logout

```

10c. Customer Panel

```

Welcome, Ahmed Ali
Sessions completed: 3

1. Browse Filter Catalog
2. Load Image
3. Build Filter Pipeline
4. Apply Pipeline & Save Result
5. View My Session History
6. Logout

```

11. Submission Checklist

Before submitting, verify every item below is working:

Image I/O

- Program reads a real JPG file correctly
- Program reads a real PNG file correctly
- Program writes a valid PNG output that opens in an image viewer
- Test pattern generator works when no image file is available

Filters

- All 10 filters are implemented and produce visually correct results
- Spatial filter work on a copy (no bleed from already-modified pixels)
- Edge pixels handled gracefully (no crash, no out-of-bounds)

OOP Concepts

- Abstract Filter base class with pure virtual methods
- All 10 filter classes inherit from Filter and override apply()
- FilterSession stores vector<Filter*>; runtime polymorphism is visible
- Image uses Pixel** with proper new/delete; deep copy constructor
- Friend class relationship between FilterSession and Image
- Method chaining on addFilter()
- static Pixel::clamp(); const getters; this pointer usage
- Multiple inheritance demonstrated with at least one class
- Virtual destructor on Filter
- User hierarchy: Admin and Customer both extend abstract User
- Constructor/destructor order visible in output
- operator+ and operator<< overloaded on Pixel
- try/catch around file I/O operations

File I/O & Persistence

- customers.txt read, written, and updated correctly
- catalog.txt toggling persists across program restarts
- sessions.txt records every completed session
- blocked_cnics.txt prevents blocked users from re-registering
- Output PNG file opens correctly in an image viewer

UI & Validation

- All menus loop correctly; Logout returns to main screen

- CNIC validated to exactly 13 digits; duplicates rejected
- Password validated (9 chars, 1 uppercase, 1 digit)
- 3-attempt login lockout works
- ASCII preview shown after each filter step

12. Helpful Hints

1. Build and test the Pixel class (clamp, operators, ASCII char).
2. Build Image with a fixed 5×5 test grid; print ASCII preview.
3. Implement JPG/PNG file reading using stb_image.
4. Implement 3–4 simple filters (Grayscale, Invert, Brightness, Flip Horizontal).
5. Build FilterSession with a hardcoded pipeline; test end-to-end.
6. Implement PNG file writing using stb_image_write; open output in a viewer to verify.
7. Add registration/login, file management, admin panel, menus.
8. Implement remaining filters; add all OOP features.

GOOD LUCK :)